

RL Homework 3: Soft Actor-Critic

Kai-Yuan Jeng A131056

kaiyuanjeng@gapp.nthu.edu.tw

May 23, 2025

Problem 1 (Soft Actor-Critic for Continuous Control)

(a) Actor Network of SAC

- In the SAC algorithm, the actor network outputs both a parameterized mean μ_ϕ and a parameterized log standard deviation $\log \sigma_\phi$, defining a Gaussian distribution over actions for each state. This enables the stochastic policy $\pi_\phi(a|s) = \mathcal{N}(\mu_\phi(s), \sigma_\phi^2(s))$, so that the agent can not only exploit high-value actions but also control its exploration dynamically. The code implements this as:

```
1         self.mu_layer = nn.Linear(128, out_dim)
2         self.mu_layer = init_layer_uniform(self.mu_layer)
3         self.log_std_layer = nn.Linear(128, out_dim)
4         self.log_std_layer = init_layer_uniform(self.
           log_std_layer)
```

This ensures the learned policy is both expressive and practical.

- The reparameterization trick is used to allow for differentiable sampling from the Gaussian policy. After computing μ_ϕ and σ_ϕ , the code transforms them into a distribution and samples as follows:

```
1         mu = self.mu_layer(x).tanh()
2         log_std = self.log_std_layer(x).tanh()
3         log_std = self.log_std_min + 0.5 * (self.log_std_max
           - self.log_std_min) * (log_std + 1)
4         std = torch.exp(log_std)
5         dist = Normal(mu, std)
6         z = dist.rsample()
```

Here, `z = dist.rsamle()` implements the reparameterization $\mathbf{u} = \mu_\phi + \sigma_\phi \odot \epsilon$, with $\epsilon \sim \mathcal{N}(0, I)$, supporting gradient backpropagation through the sampling process. This trick allows gradients to flow from the loss function back to the policy network parameters.

- Since $\mathbf{u}$ (`z`) is an unbounded sample from a Gaussian, we squash it using $\tanh$ to create an action $a = \tanh(\mathbf{u})$ (`action = z.tanh()`) that lies in the valid interval $(-1, 1)$. This non-linear mapping alters the probability density, so we must apply the change-of-variables formula to correctly compute the log-probability of the bounded action (see Appendix C of [1]). For an action a with support in $(-1, 1)^D$ (a D -dimensional action space),

$$\pi(a \mid s) = \mathcal{N}(\mathbf{u}; \mu_\phi(s), \sigma_\phi^2(s)) \left| \det \left(\frac{da}{d\mathbf{u}} \right) \right|^{-1}, \quad (1)$$

where $a = \tanh(\mathbf{u})$, $\mathbf{u} \sim \mathcal{N}(\mu_\phi, \sigma_\phi^2)$. The Jacobian is $\frac{da}{d\mathbf{u}} = \text{diag}(1 - \tanh^2(\mathbf{u}))$. Thus, the log-likelihood becomes:

$$\log \pi(a \mid s) = \log \mathcal{N}(\mathbf{u}; \mu_\phi(s), \sigma_\phi^2(s)) - \sum_{i=1}^D \log(1 - \tanh^2(u_i)) \quad (2)$$

with u_i being the i -th element of $\mathbf{u}$. This process is implemented via:

```

1      action = z.tanh()
2      log_prob = dist.log_prob(z) - torch.log(1 - action.
      pow(2) + 1e-7)
3      log_prob = log_prob.sum(-1, keepdim=True)
```

The small constant $1e - 7$ is added for numerical stability. This correction is essential for computing accurate policy gradients in bounded continuous action spaces. This step guarantees that the policy update is mathematically correct under the stochastic, bounded action setting.

(b) Soft Q Network of SAC

- Soft Actor-Critic (SAC) adopts two independent soft Q-networks (“twin Q”) as its critics. Each Q-network estimates the expected cumulative return for a state-action pair (s, a) . A well-known problem in value-based reinforcement learning is the overestimation bias of Q-values, which arises when using a single Q-network with the max operator during target calculation. To address this, SAC (as in TD3 [2]) employs

two Q-networks and, during target computation, uses the minimum of their two values (e.g., `q_hat = torch.min(new_q_1, new_q_2)`). This min operator significantly reduces positive bias and stabilizes learning.

- Formally, the loss function for updating each of the two soft Q-networks is

$$\mathcal{L}_{Q_{\theta_j}} = \frac{1}{N} \sum_{i=1}^N \left(Q_{\theta_j}(s_i, a_i) - \left(r_i + \gamma \mathbb{E}_{a' \sim \pi_\phi(\cdot | s'_i)} \left[\hat{Q}_\theta(s'_i, a') - \alpha \log \pi_\phi(a' | s'_i) \right] \right) \right)^2, \quad (3)$$

with

$$\hat{Q}_\theta(s, a) = \min_{j \in \{1, 2\}} Q_{\theta_j}(s, a), \quad \forall (s, a) \in \mathcal{S} \times \mathcal{A}, \quad (4)$$

where N is the mini-batch size, (s_i, a_i, r_i, s'_i) is a sample from the replay buffer, γ is the discount factor, θ_j are the parameters of the Q-networks, α is the entropy temperature (for exploration), and $a' \sim \pi_\phi(\cdot | s'_i)$ denotes a sample from the current policy. The expectation is typically approximated by a single Monte Carlo sample in actual code.

In some SAC implementations, including the provided codebase, an additional value network V_ψ with a target $V_{\bar{\psi}}$ is used for target calculation. The loss then becomes:

$$\mathcal{L}_{Q_{\theta_j}} = \frac{1}{N} \sum_{i=1}^N \left(Q_{\theta_j}(s_i, a_i) - (r_i + \gamma V_{\bar{\psi}}(s'_i)) \right)^2, \quad j \in \{1, 2\}, \quad (5)$$

where $V_{\bar{\psi}}$ is the slowly updated target value network. The target consists of the immediate reward r_i plus the discounted value of the next state s'_i .

(c) Alpha Network of SAC

- The entropy temperature α is optimized through

$$\mathcal{L}_\alpha = -\alpha \cdot (\log \pi(a | s) + \mathcal{H}), \quad (6)$$

where $\log \pi(a | s)$ is computed from the current policy (see Eq. 2). This is implemented as:

```
1 alpha_loss = (-self.log_alpha.exp() * (log_prob + self.
    target_entropy).detach()).mean()
```

The intuition behind automating the entropy is that the policy should explore more in uncertain regions while being more deterministic in states where good and bad actions

are clearly distinguished [3].

Suppose we define an entropy threshold $\mathcal{H}$ as the desired minimum expected entropy. The occupancy measure ρ_π is the (discounted) visitation probability of (s_t, a_t) under policy π :

$$\rho_\pi(s_t, a_t) = P(s_t, a_t \mid \pi). \quad (7)$$

Then, finding a stochastic policy with maximal expected return that satisfies a minimum expected entropy constraint is formulated as:

$$\max_{\pi_{0:T}} \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} \left[\sum_{t=0}^T r(s_t, a_t) \right] \quad \text{s.t.} \quad \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} [-\log \pi_t(a_t \mid s_t)] \geq \mathcal{H} \quad \forall t. \quad (8)$$

Note that this constrained optimization problem allows for a sequence of non-stationary policies $\pi_{0:T}$ —that is, the optimal policy at each time step t may be different.

As suggested in [3], the objective

$$\max_{\pi_{0:T}} \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} \left[\sum_{t=0}^T r(s_t, a_t) \right] \quad (9)$$

can be rewritten as an iterated or backward maximization at each time step t :

$$\max_{\pi_0} \mathbb{E}_{a_0 \sim \pi_0(\cdot \mid s_0)} \left[r(s_0, a_0) + \max_{\pi_1} \mathbb{E}_{a_1 \sim \pi_1(\cdot \mid s_1)} [r(s_1, a_1) + \dots + \max_{\pi_T} \mathbb{E}_{a_T \sim \pi_T(\cdot \mid s_T)} [r(s_T, a_T)]] \right] \quad (10)$$

with the entropy constraint enforced at each time step t .

Consider the final time step T . The decomposed constrained optimization at T becomes the following primal problem:

$$\max_{\pi_T} \mathbb{E}_{a_T \sim \pi_T(\cdot \mid s_T)} [r(s_T, a_T)] \quad \text{s.t.} \quad \mathbb{E}_{a_T \sim \pi_T(\cdot \mid s_T)} [-\log \pi_T(a_T \mid s_T)] \geq \mathcal{H}. \quad (11)$$

If the target entropy satisfies

$$\mathcal{H} < \log |\mathcal{A}|, \quad (12)$$

where $\mathcal{A}$ is the discrete action space, then any uniformly random policy $\pi_T^{\text{unif}}(a_T \mid s_T) = 1/|\mathcal{A}|$ will have entropy $\log |\mathcal{A}| > \mathcal{H}$, and hence is a strictly feasible solution. As the objective is linear and the constraint is convex in π_T , strong duality holds by Slater's

condition.

The corresponding dual (Lagrangian) problem is:

$$\min_{\alpha_T \geq 0} \max_{\pi_T} \mathbb{E}_{a_T \sim \pi_T(\cdot | s_T)} [r(s_T, a_T) - \alpha_T \log \pi_T(a_T | s_T) - \alpha_T \mathcal{H}], \quad (13)$$

where α_T is the dual variable (Lagrange multiplier), interpreted as the entropy temperature.

The optimal dual variable can then be obtained via the optimal policy π_T^* as:

$$\alpha_T^* = \arg \min_{\alpha_T \geq 0} \mathbb{E}_{a_T \sim \pi_T^*(\cdot | s_T)} [-\alpha_T \log \pi_T^*(a_T | s_T; \alpha_T) - \alpha_T \mathcal{H}] \quad (14)$$

since $r(s_T, a_T)$ is independent of α_T and does not affect the minimization.

This entire line of reasoning applies iteratively at every time step t . In practice, since the optimal policy π^* is unknown and being learned concurrently, we approximate the expectation in the dual loss by sampling from the current policy parameters. Thus, at each update step, α is optimized using:

$$\mathcal{L}_\alpha = -\alpha \cdot (\log \pi(a | s) + \mathcal{H}), \quad (15)$$

where $\log \pi(a | s)$ is computed from the current policy (see Eq. 2). This approximation is valid as both the policy and the entropy temperature are updated iteratively and jointly during training. In code, this is implemented as:

```
1 alpha_loss = (-self.log_alpha.exp() * (log_prob + self.
    target_entropy).detach()).mean()
```

In summary, the reason for implementing α auto-tuning as above is that, by minimizing this loss with respect to α , the policy entropy is adaptively driven toward the target value $\mathcal{H}$ throughout training.

(d) SAC on Pendulum-v1

- **Q-function Loss:** The critic is trained by minimizing the Bellman error with the soft (entropy-adjusted) target:

```
1 with torch.no_grad():
2     v_target = self.vf_target(next_state)
3     y = reward + mask * self.gamma * v_target
4
```

```
5     q_1 = self.qf_1(state, action)
6     q_2 = self.qf_2(state, action)
7     qf_1_loss = F.mse_loss(q_1, y)
8     qf_2_loss = F.mse_loss(q_2, y)
```

- **Value-function Loss:** The value network is trained to match the expected soft Q-value under current policy:

```
1     new_q_1 = self.qf_1(state, new_action)
2     new_q_2 = self.qf_2(state, new_action)
3     q_hat = torch.min(new_q_1, new_q_2)
4     v = self.vf(state)
5     vf_loss = F.mse_loss(v, (q_hat - alpha * log_prob).detach())
```

- **Actor Loss:** The policy (actor) is updated to maximize expected return and entropy:

```
1     actor_loss = (alpha * log_prob - q_hat).mean()
```

Neural Network Structure and Hyperparameters

- **Actor/Critic/Value:** 2 hidden layers, each with 128 units, ReLU activation
- **Learning rate:** 3×10^{-4}
- **Discount factor (γ):** 0.99
- **Target smoothing (τ):** 5×10^{-3}
- **Batch size:** 256
- **Replay buffer:** 1,000,000 transitions
- **Initial random steps:** 1,000
- **Total steps:** 100,000
- **Policy update frequency:** 1
- **Random seed:** 77

Results and Learning Dynamics

- Achieved average return: **-117.8** (solved **>-130** after $\sim 20k$ steps)
- Final losses:
 - Q loss: **5.39**
 - V loss: **2.29**
 - Actor loss: **25.96**
 - Alpha loss: **-0.0028**
- Training curves (see figures below).

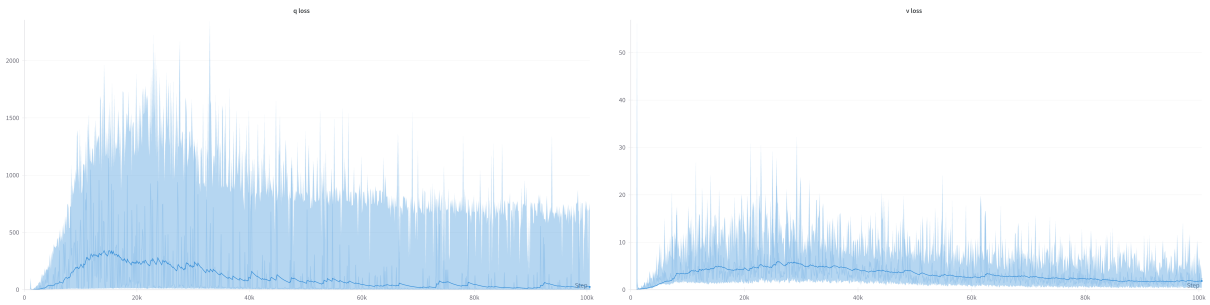


Figure 1: (Left) Q Loss; (Right) V Loss during training on Pendulum-v1

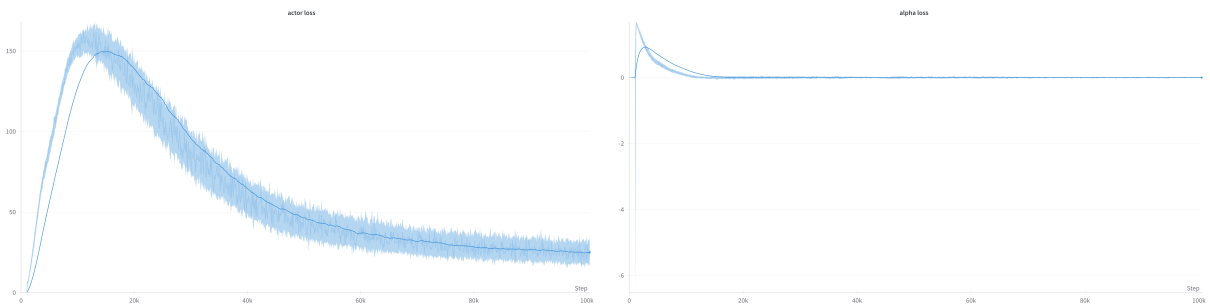


Figure 2: (Left) Actor Loss; (Right) Alpha Loss during training on Pendulum-v1

(e) SAC on HalfCheetah-v5

Hyperparameter Search (Weights & Biases Sweep) Performance in HalfCheetah-v5 is more sensitive to hyperparameter choices than in simple environments. Therefore, we conducted a hyperparameter sweep using Weights & Biases (W&B) `sweep.yaml` with Bayesian optimization over the following ranges:

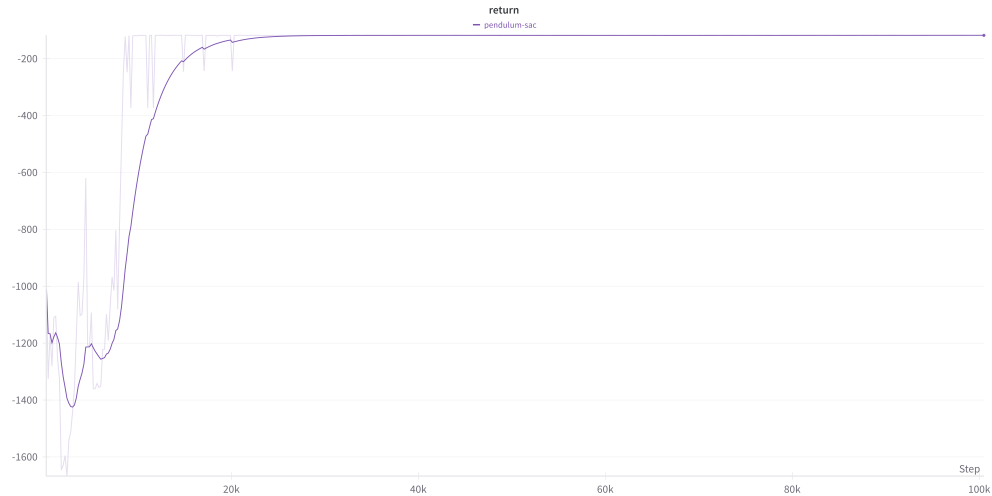


Figure 3: Pendulum-v1 average episodic return.

```

1 program: sac_halfcheetah.py
2 method: bayes
3 name: halfcheetah-sac-sweep
4 metric:
5     name: combined_score
6     goal: maximize
7 parameters:
8     lr:
9         distribution: uniform
10        min: 0.0001
11        max: 0.001
12    tau:
13        distribution: uniform
14        min: 0.001
15        max: 0.01
16    discount-factor:
17        values: [0.98, 0.99, 0.995]

```

We devised a custom sweep metric ("combined_score") to favor both fast learning and high late returns:

```

1 N = 100
2 early_avg = np.mean(scores[:N])
3 late_avg = np.mean(scores[-N:])
4 slope = ((late_avg - early_avg) / (len(scores) - N)) * 100
5 combined_score = 0.7 * score + 0.3 * slope

```

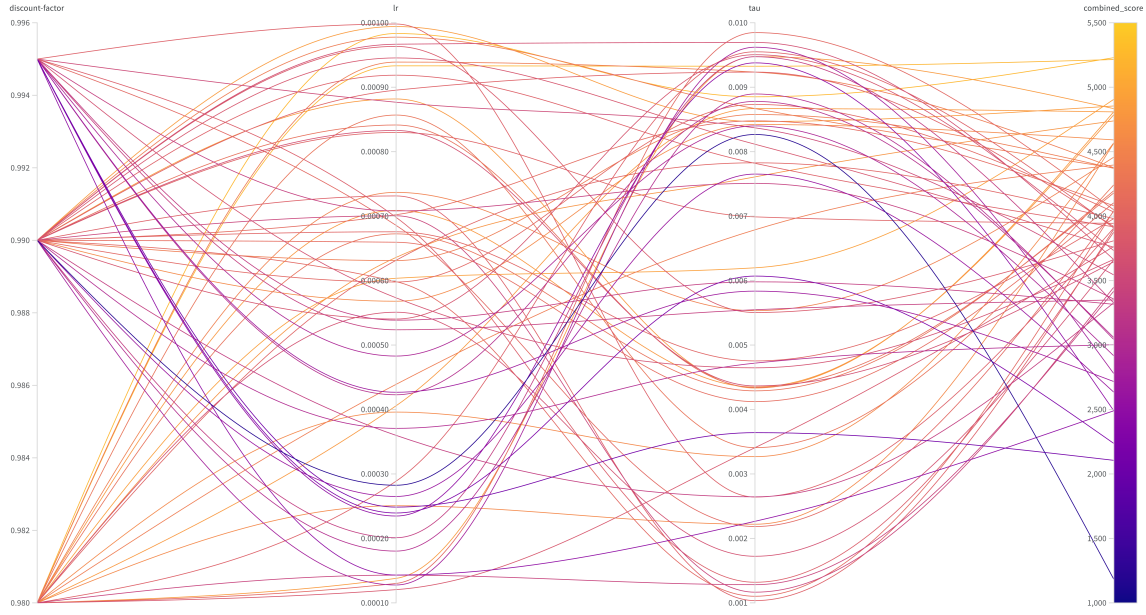


Figure 4: Sweep Result

This rewards agents that not only reach high scores but also improve rapidly. The results are shown in Fig. 4.

Neural Network Structure and Hyperparameters Based on sweep results and SAC best practices [3]:

- **Actor/Critic/Value:** 2 hidden layers, 512 units each, ReLU activation
- **Learning rate:** 6.34×10^{-4}
- **Discount factor (γ):** 0.98
- **Target smoothing (τ):** 2.24×10^{-3}
- **Batch size:** 512
- **Replay buffer:** 1,000,000
- **Initial random steps:** 10,000
- **Total steps:** 1,000,000
- **Policy update frequency:** 1
- **Random seed:** 77

Results and Learning Dynamics

- Achieved average return: **7,490** (solved **>7000**)
- Final losses:
 - Q loss: **408.08**
 - V loss: **4.43**
 - Actor loss: **-275.02**
 - Alpha loss: **0.0048**
- Training curves (see figures below).

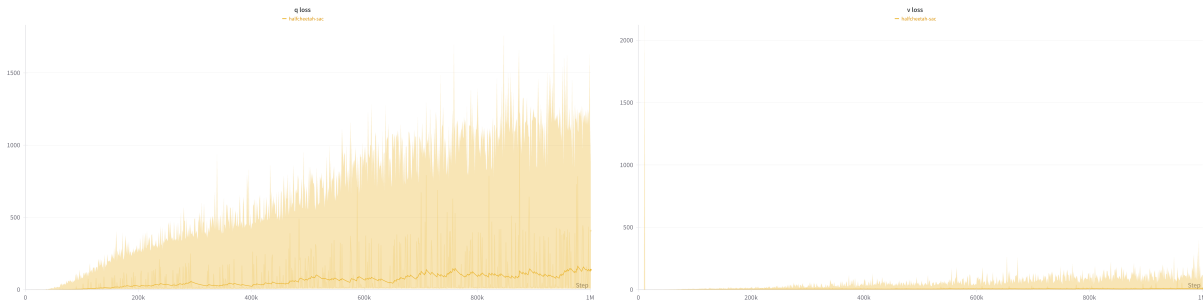


Figure 5: (Left) Q Loss; (Right) V Loss during training on HalfCheetah-v5

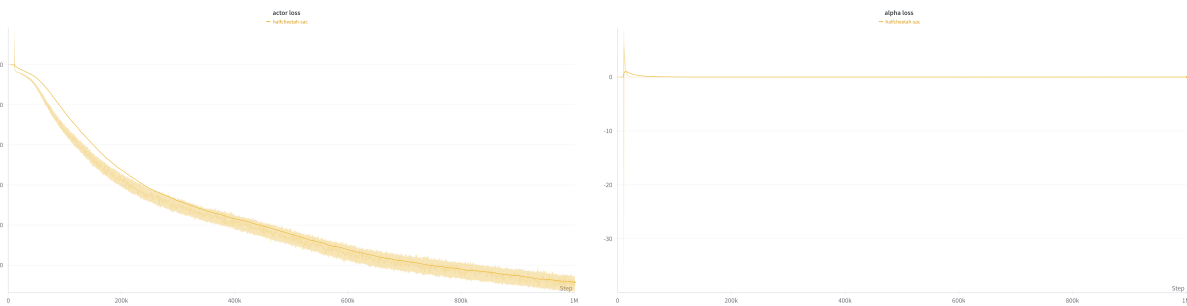


Figure 6: (Left) Actor Loss; (Right) Alpha Loss during training on HalfCheetah-v5

References

- [1] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” 2018. [Online]. Available: <https://arxiv.org/abs/1801.01290>

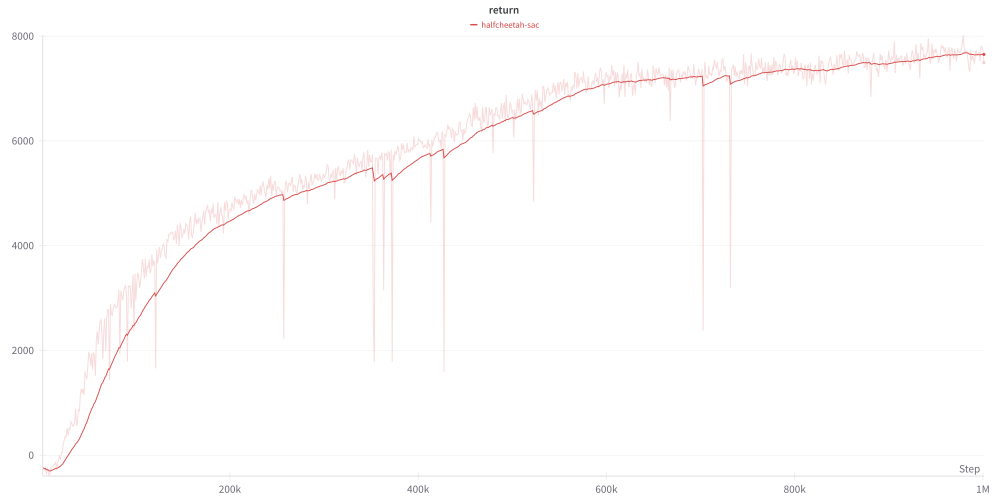


Figure 7: HalfCheetah-v5 average episodic return.

- [2] S. Fujimoto, H. van Hoof, and D. Meger, “Addressing function approximation error in actor-critic methods,” 2018. [Online]. Available: <https://arxiv.org/abs/1802.09477>
- [3] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, and S. Levine, “Soft actor-critic algorithms and applications,” 2019. [Online]. Available: <https://arxiv.org/abs/1812.05905>